In chapter 6 you learned that authenticated encryption can be used to provide the benefits of encryption to hide confidential attributes and authentication to ensure that a JWT is genuine and has not been tampered with. Encrypted JWTs can be useful for access tokens too, because the AS may want to include attributes in the access token that are useful for the API for making access control decisions, but which should be kept confidential from third-party clients or from the user themselves. For example, the AS may include the resource owner's email address in the token for use by the API, but this information should not be leaked to the third-party client. In this case the AS can encrypt the access token JWT using an encryption key that only the API can decrypt.

Unfortunately, none of the public key encryption algorithms supported by the JWT standards provide authenticated encryption,[10] because this is less often implemented for public key cryptography. The supported algorithms provide only confidentiality and so must be combined with a digital signature to ensure the JWT is not tampered with or forged. This is done by first signing the claims to produce a signed JWT, and then encrypting that signed JWT to produce a nested JOSE structure (figure 7.9). The downside is that the resulting JWT is much larger than it would be if it was just signed and requires two expensive public key operations to first decrypt the outer encrypted JWE and then verify the inner signed JWT. You shouldn't use the same key for encryption and signing, even if the algorithms are compatible.